

```

--
=====
=====
-- SECTION: FUNCTIONS AND PROCEDURES (public
schema) -- 67 objects
-- Exact bodies via pg_get_functiondef(). Includes
hg_is_admin() and every
-- other hg_* function your Python backend calls
via RPC or that your
-- triggers/RLS policies depend on. Apply AFTER 01-
04 (tables/constraints)
-- since some function bodies reference tables.
--
=====
=====

-- ----- checkin_event_atomic -----
CREATE OR REPLACE FUNCTION
public.checkin_event_atomic(p_event_id uuid,
p_qr_token text, p_user_id uuid)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
    DECLARE
        v_ticket_id UUID;
        v_checkin_id UUID;
    BEGIN
        -- Find user's ticket for this event
        SELECT id INTO v_ticket_id
        FROM event_tickets
        WHERE event_id = p_event_id
        AND user_id = p_user_id
        LIMIT 1;

```

```

                IF NOT FOUND THEN
                    RETURN
jsonb_build_object('error', 'No ticket found for
this event');
                END IF;

                -- QR token must
match the ticket UUID
                IF
v_ticket_id::TEXT <> p_qr_token THEN
                    RETURN
jsonb_build_object('error', 'Invalid QR token');
                    END IF;

                --
Prevent double check-in
                IF
EXISTS (SELECT 1 FROM event_checkins WHERE
ticket_id = v_ticket_id) THEN

RETURN jsonb_build_object('error', 'Already checked
in to this event');

END IF;

-- Record check-in

INSERT INTO event_checkins (ticket_id,
checked_in_at)

VALUES (v_ticket_id, now())

RETURNING id INTO v_checkin_id;

RETURN jsonb_build_object(

'success',    TRUE,

```

```

        'checkin_id', v_checkin_id,

        'ticket_id', v_ticket_id

    );

    END;

    $function$;

-- ----- claim_guest_order -----
CREATE OR REPLACE FUNCTION
public.claim_guest_order(p_order_id uuid,
p_claim_token uuid, p_user_id uuid)
    RETURNS jsonb
    LANGUAGE plpgsql
    SECURITY DEFINER
    SET search_path TO 'public'
AS $function$
DECLARE
    v_order RECORD;
BEGIN
    -- Object Ownership/Spoofing check
    IF auth.uid() IS NOT NULL AND auth.uid() <>
p_user_id AND (SELECT role FROM public.profiles
WHERE id = auth.uid()) NOT IN ('admin'::user_role,
'super_admin'::user_role, 'superadmin'::user_role)
    THEN
        RAISE EXCEPTION 'claim_guest_order: Cannot
claim order for another user.';
    END IF;

    -- Find the guest order by ID and claim token
    SELECT id, user_id, status, total_amount,
claim_token
    INTO    v_order
    FROM    orders
    WHERE   id            = p_order_id
           AND claim_token = p_claim_token
    LIMIT 1;

```

```

    IF NOT FOUND THEN
        RETURN jsonb_build_object('error', 'Order not
found or claim token invalid');
    END IF;

    IF v_order.user_id IS NOT NULL THEN
        RETURN jsonb_build_object('error', 'Order is
already linked to an account');
    END IF;

    -- Link the order to the registered user and
clear the claim token
    UPDATE orders
    SET     user_id      = p_user_id,
           claim_token = NULL
    WHERE  id = p_order_id;

    RETURN jsonb_build_object(
        'success',      TRUE,
        'order_id',     p_order_id,
        'user_id',      p_user_id,
        'status',       v_order.status,
        'total_amount', v_order.total_amount
    );
END;
$function$;

-- ----- credit_wallet_atomic -----
CREATE OR REPLACE FUNCTION
public.credit_wallet_atomic(p_user_id uuid,
p_amount numeric, p_reason text, p_reference_type
text, p_reference_id uuid, p_provider text,
p_provider_reference text, p_metadata jsonb)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE

```

```

v_balance NUMERIC;
v_txn_id UUID;
BEGIN
-- Idempotency check:
IF p_reference_type IS NOT NULL AND
p_reference_id IS NOT NULL THEN
    SELECT id, balance_after INTO v_txn_id,
v_balance
    FROM public.wallet_transactions
    WHERE user_id = p_user_id AND reference_type =
p_reference_type AND reference_id = p_reference_id
AND type = 'credit';
    IF FOUND THEN
        RETURN jsonb_build_object('success', true,
'new_balance', v_balance, 'transaction_id',
v_txn_id, 'replayed', true);
    END IF;
END IF;

IF p_provider_reference IS NOT NULL AND
p_provider IS NOT NULL THEN
    SELECT id, balance_after INTO v_txn_id,
v_balance
    FROM public.wallet_transactions
    WHERE user_id = p_user_id AND provider =
p_provider AND provider_reference =
p_provider_reference AND type = 'credit';
    IF FOUND THEN
        RETURN jsonb_build_object('success', true,
'new_balance', v_balance, 'transaction_id',
v_txn_id, 'replayed', true);
    END IF;
END IF;

-- Row-lock wallet
SELECT balance INTO v_balance FROM public.wallets
WHERE user_id = p_user_id FOR UPDATE;
IF NOT FOUND THEN
    INSERT INTO public.wallets (user_id, balance,
currency)

```

```

VALUES (p_user_id, p_amount, 'NGN')
RETURNING balance INTO v_balance;
ELSE
    v_balance := v_balance + p_amount;
    UPDATE public.wallets SET balance = v_balance,
updated_at = now() WHERE user_id = p_user_id;
END IF;

-- Insert wallet ledger transaction
INSERT INTO public.wallet_transactions (
    user_id, type, amount, balance_after, reason,
reference_type, reference_id, provider,
provider_reference, metadata
)
VALUES (
    p_user_id, 'credit', p_amount, v_balance,
p_reason, p_reference_type, p_reference_id,
p_provider, p_provider_reference, p_metadata
)
RETURNING id INTO v_txn_id;

RETURN jsonb_build_object('success', true,
'new_balance', v_balance, 'transaction_id',
v_txn_id);
END;
$function$;

-- ----- debit_wallet_atomic -----
CREATE OR REPLACE FUNCTION
public.debit_wallet_atomic(p_user_id uuid, p_amount
numeric, p_reason text, p_reference_type text,
p_reference_id uuid, p_metadata jsonb)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_balance NUMERIC;
    v_txn_id UUID;

```

```

BEGIN
    -- Idempotency check:
    IF p_reference_type IS NOT NULL AND
    p_reference_id IS NOT NULL THEN
        SELECT id, balance_after INTO v_txn_id,
        v_balance
        FROM public.wallet_transactions
        WHERE user_id = p_user_id AND reference_type =
        p_reference_type AND reference_id = p_reference_id
        AND type = 'debit';
        IF FOUND THEN
            RETURN jsonb_build_object('success', true,
            'new_balance', v_balance, 'transaction_id',
            v_txn_id, 'replayed', true);
        END IF;
    END IF;

    -- Row-lock wallet
    SELECT balance INTO v_balance FROM public.wallets
    WHERE user_id = p_user_id FOR UPDATE;
    IF NOT FOUND THEN
        RETURN jsonb_build_object('error', 'Wallet not
        found for user');
    END IF;

    IF v_balance < p_amount THEN
        RETURN jsonb_build_object('error',
        'Insufficient wallet balance');
    END IF;

    v_balance := v_balance - p_amount;
    UPDATE public.wallets SET balance = v_balance,
    updated_at = now() WHERE user_id = p_user_id;

    -- Insert wallet ledger transaction
    INSERT INTO public.wallet_transactions (
        user_id, type, amount, balance_after, reason,
        reference_type, reference_id, metadata
    )
    VALUES (

```

```

        p_user_id, 'debit', p_amount, v_balance,
        p_reason, p_reference_type, p_reference_id,
        p_metadata
    )
    RETURNING id INTO v_txn_id;

    RETURN jsonb_build_object('success', true,
        'new_balance', v_balance, 'transaction_id',
        v_txn_id);
END;
$function$;

-- ----- get_menu_item_review_stats -----
CREATE OR REPLACE FUNCTION
public.get_menu_item_review_stats(p_item_ids
uuid[])
RETURNS TABLE(menu_item_id uuid, avg_rating
numeric, review_count bigint)
LANGUAGE sql
STABLE
SET search_path TO 'public'
AS $function$
    WITH item_orders AS (
        SELECT DISTINCT oi.menu_item_id,
oi.order_id
        FROM public.order_items oi
        WHERE oi.menu_item_id = ANY(p_item_ids)
    )
    SELECT
        mi.id,
        COALESCE(ROUND(AVG(r.rating)::numeric, 1),
0::numeric),
        COUNT(DISTINCT r.id)
    FROM public.menu_items mi
    LEFT JOIN item_orders io
        ON io.menu_item_id = mi.id
    LEFT JOIN public.orders o
        ON o.id = io.order_id
        AND o.status = 'delivered'
    LEFT JOIN public.order_reviews r

```



```

        ON r.order_id = o.id
        AND r.rating IS NOT NULL
        WHERE mi.id = ANY(p_item_ids)
        GROUP BY mi.id;
$function$;

-- ----- hg_add_hp_transaction -----
CREATE OR REPLACE FUNCTION
public.hg_add_hp_transaction(p_user_id uuid, p_type
text, p_amount integer, p_source text,
p_reference_type text DEFAULT NULL::text,
p_reference_id uuid DEFAULT NULL::uuid,
p_issued_by_admin_id uuid DEFAULT NULL::uuid,
p_metadata jsonb DEFAULT '{}'::jsonb, p_expires_at
timestamp with time zone DEFAULT NULL::timestamp
with time zone)
RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_current integer;
    v_new_balance integer;
    v_delta integer;
    v_transaction_id uuid;
begin
    if p_amount <= 0 then
        raise exception 'HP amount must be positive';
    end if;

    if p_type not in ('earn', 'spend', 'expire',
'adjustment') then
        raise exception 'Invalid HP transaction type:
%', p_type;
    end if;

    select hp_balance into v_current
    from profiles
    where id = p_user_id

```

```

for update;

if v_current is null then
    raise exception 'Profile not found for user %',
p_user_id;
end if;

if p_type in ('earn', 'adjustment') then
    v_delta := p_amount;
else
    v_delta := -p_amount;
end if;

v_new_balance := v_current + v_delta;
if v_new_balance < 0 then
    raise exception 'Insufficient HP balance';
end if;

update profiles
set hp_balance = v_new_balance,
    last_hp_activity_at = case when p_type =
'earn' then now() else last_hp_activity_at end,
    updated_at = now()
where id = p_user_id;

insert into hp_transactions (
    user_id, type, amount, balance_after, source,
reference_type, reference_id,
    issued_by_admin_id, metadata, expires_at,
remaining_amount
) values (
    p_user_id, p_type, p_amount, v_new_balance,
p_source, p_reference_type, p_reference_id,
    p_issued_by_admin_id, coalesce(p_metadata,
'{}'::jsonb), p_expires_at,
    case when p_type = 'earn' then p_amount else
null end
) returning id into v_transaction_id;

perform

```

```

public.hg_set_profile_tier_from_balance(p_user_id);

    return v_transaction_id;
end;
$function$;

-- ----- hg_add_wallet_transaction -----
CREATE OR REPLACE FUNCTION
public.hg_add_wallet_transaction(p_user_id uuid,
p_type text, p_amount numeric, p_reason text
DEFAULT NULL::text, p_reference_type text DEFAULT
NULL::text, p_reference_id uuid DEFAULT NULL::uuid,
p_provider text DEFAULT NULL::text,
p_provider_reference text DEFAULT NULL::text,
p_issued_by_admin_id uuid DEFAULT NULL::uuid,
p_metadata jsonb DEFAULT '{}'::jsonb)
RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_current numeric(14,2);
    v_new_balance numeric(14,2);
    v_delta numeric(14,2);
    v_transaction_id uuid;
begin
    if p_amount <= 0 then
        raise exception 'Wallet amount must be
positive';
    end if;

    if p_type not in ('credit', 'debit', 'refund',
'adjustment') then
        raise exception 'Invalid wallet transaction
type: %', p_type;
    end if;

    insert into wallets (user_id, balance, currency)
values (p_user_id, 0, 'NGN')

```

```

on conflict (user_id) do nothing;

select balance into v_current
from wallets
where user_id = p_user_id
for update;

if p_type in ('credit', 'refund', 'adjustment')
then
    v_delta := p_amount;
else
    v_delta := -p_amount;
end if;

v_new_balance := v_current + v_delta;
if v_new_balance < 0 then
    raise exception 'Insufficient wallet balance';
end if;

update wallets
set balance = v_new_balance,
    updated_at = now()
where user_id = p_user_id;

update profiles
set wallet_balance = v_new_balance,
    updated_at = now()
where id = p_user_id;

insert into wallet_transactions (
    user_id, type, amount, balance_after, reason,
reference_type, reference_id,
    provider, provider_reference,
issued_by_admin_id, metadata
) values (
    p_user_id, p_type, p_amount, v_new_balance,
p_reason, p_reference_type, p_reference_id,
    p_provider, p_provider_reference,
p_issued_by_admin_id, coalesce(p_metadata,
'{}'::jsonb)

```

```

    ) returning id into v_transaction_id;

    return v_transaction_id;
end;
$function$;

-- ----- hg_anonymize_user -----
CREATE OR REPLACE FUNCTION
public.hg_anonymize_user(p_user_id uuid)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    actor_role user_role;
    v_exists BOOLEAN;
BEGIN
    -- Verify caller is super_admin or the user
    themselves (account deletion/anonymization
    requests)
    SELECT EXISTS(SELECT 1 FROM public.profiles WHERE
    id = auth.uid() AND role =
    'super_admin'::user_role) INTO v_exists;
    IF NOT v_exists AND auth.uid() <> p_user_id THEN
        RETURN jsonb_build_object('success', false,
    'error', 'Unauthorized to perform anonymization on
    this user');
    END IF;

    -- Verify target user exists
    SELECT EXISTS(SELECT 1 FROM public.profiles WHERE
    id = p_user_id) INTO v_exists;
    IF NOT v_exists THEN
        RETURN jsonb_build_object('success', false,
    'error', 'User not found');
    END IF;

    -- 1. Anonymize profiles table (Scrub all PII)
    UPDATE public.profiles

```

```

SET
    email = ('deleted-' ||
substring(p_user_id::text, 1, 8) ||
'@deleted.holygrills.com.ng')::extensions.citext,
    full_name = 'Anonymized Customer',
    phone = NULL,
    date_of_birth = NULL,
    photo_url = NULL,
    preferences = '{} '::jsonb,
    is_active = FALSE,
    deactivated_at = now(),
    deactivation_reason = 'User Requested Account
Deletion / Anonymization',
    onboarding_completed_at = NULL,
    referral_code = NULL,
    last_seen_at = NULL,
    last_activity_at = NULL,
    push_enabled = FALSE,
    email_notifications = FALSE,
    updated_at = now()
WHERE id = p_user_id;

-- 2. Anonymize user_addresses table
UPDATE public.user_addresses
SET
    line1 = 'Redacted Line 1',
    line2 = NULL,
    landmark = NULL,
    latitude = NULL,
    longitude = NULL,
    is_default = FALSE,
    updated_at = now()
WHERE user_id = p_user_id;

-- 3. Anonymize push_subscriptions
DELETE FROM public.push_subscriptions WHERE
user_id = p_user_id;

-- 4. Anonymize device_tokens
DELETE FROM public.device_tokens WHERE user_id =

```

```

p_user_id;

-- Log in audit trail if an admin was the actor
IF auth.uid() <> p_user_id THEN
    SELECT role INTO actor_role FROM
public.profiles WHERE id = auth.uid();
    INSERT INTO public.admin_audit_logs (
        action, entity_type, entity_id, actor_id,
actor_role, after_value
    ) VALUES (
        'anonymize_user', 'profiles',
p_user_id::text, auth.uid(), actor_role::text,
        jsonb_build_object('anonymized_at', now()),
'requested_by', 'admin')
    );
END IF;

RETURN jsonb_build_object('success', true,
'user_id', p_user_id, 'anonymized_at', now());
END;
$function$;

-- ----- hg_auto_credit_hp_on_delivery -----
---
CREATE OR REPLACE FUNCTION
public.hg_auto_credit_hp_on_delivery()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    IF NEW.status = 'delivered' AND OLD.status !=
'delivered'
        AND NEW.hp_credited_at IS NULL AND NEW.user_id
IS NOT NULL THEN
        PERFORM public.hg_credit_order_hp(NEW.id);
    END IF;
    RETURN NEW;
END;

```

```

$function$;

-- ----- hg_auto_referral_code -----
CREATE OR REPLACE FUNCTION
public.hg_auto_referral_code()
  RETURNS trigger
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
AS $function$
begin
  if new.referral_code is null or new.referral_code
= '' then
    new.referral_code :=
public.hg_generate_referral_code(coalesce(new.email
::text, gen_random_uuid()::text));
  end if;
  return new;
end;
$function$;

-- ----- hg_create_wallet_on_profile -----
-
CREATE OR REPLACE FUNCTION
public.hg_create_wallet_on_profile()
  RETURNS trigger
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
AS $function$
BEGIN
  INSERT INTO public.wallets (user_id, balance,
currency, created_at, updated_at)
  VALUES (NEW.id, COALESCE(NEW.wallet_balance, 0),
'NGN', NOW(), NOW())
  ON CONFLICT (user_id) DO NOTHING;
  RETURN NEW;
END;
$function$;

```



```

-- ----- hg_credit_order_hp -----
CREATE OR REPLACE FUNCTION
public.hg_credit_order_hp(p_order_id uuid)
RETURNS orders
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_order orders%rowtype;
    v_profile profiles%rowtype;
    v_total_hp integer := 0;
    v_item order_items%rowtype;
    v_hp_rate numeric := 1.0;
begin
    select * into v_order from orders where id =
p_order_id for update;
    if not found then
        raise exception 'Order not found: %',
p_order_id;
    end if;

    if v_order.hp_credited_at is not null then
        return v_order;
    end if;

    if v_order.user_id is null or
v_order.payment_status <> 'paid' then
        return v_order;
    end if;

    select * into v_profile from profiles where id =
v_order.user_id;
    if v_profile.current_tier_id is not null then
        select coalesce(earn_multiplier, 1) into
v_hp_rate from hp_tiers where id =
v_profile.current_tier_id;
    end if;

    for v_item in select * from order_items where

```

```

order_id = p_order_id loop
    v_total_hp := v_total_hp +
round(v_item.hp_earn_snapshot * v_item.quantity *
v_hp_rate);
    end loop;

    if v_total_hp <= 0 then
        return v_order;
    end if;

    perform public.hg_add_hp_transaction(
        v_order.user_id,
        'earn',
        v_total_hp,
        'order_delivery',
        'order',
        p_order_id,
        null,
        jsonb_build_object('hp_rate', v_hp_rate),
        now() + interval '12 months'
    );

    update orders
    set hp_earned = v_total_hp,
        hp_credited_at = now(),
        updated_at = now()
    where id = p_order_id
    returning * into v_order;

    return v_order;
end;
$function$;

-- ----- hg_current_user_role -----
CREATE OR REPLACE FUNCTION
public.hg_current_user_role()
RETURNS user_role
LANGUAGE sql
STABLE
SET search_path TO 'public'

```

```
AS $function$
    SELECT role FROM public.profiles WHERE id =
auth.uid() LIMIT 1;
$function$;
```

```
-- ----- hg_deactivate_user -----
CREATE OR REPLACE FUNCTION
public.hg_deactivate_user(target_user_id uuid,
reason text DEFAULT NULL::text)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
```

```
DECLARE
```

```
actor_role user_role;
```

```
target
public.profiles%ROWTYPE;

BEGIN

-- Verify caller is
super_admin

SELECT role INTO
actor_role

FROM
```

```
public.profiles
```

```
auth.uid();
```

```
WHERE id =
```

```
<> 'super_admin' THEN
```

```
IF actor_role
```

```
jsonb_build_object(
```

```
RETURN
```

```
'success', false,
```

```
'error', 'Only super_admin can deactivate users'
```

```
);
```

```
END IF;
```

```
-- Get target user
```

```
SELECT * INTO target
```

```
FROM public.profiles
```

```
WHERE id = target_user_id;
```

```
IF NOT FOUND THEN
```

```
RETURN jsonb_build_object(
```

```
  'success', false,
```

```
  'error', 'User not  
found'
```



```
);
```

```
END IF;
```

```
target.is_active = false THEN      IF
```

```
jsonb_build_object(                RETURN
```

```
'success', false,
```

```
'error', 'User is already deactivated'
```

```
);
```

```
END IF;
```

```
-- Soft delete
```

```
UPDATE public.profiles SET
```

```
is_active = false,
```

```
deactivated_at =  
now(),
```

```
deactivated_by =  
auth.uid(),
```

```
deactivation_reason = reason,
```

```
updated_at = now()
```

```
WHERE id =  
target_user_id;
```

```
-- Log in  
audit trail
```

```
INSERT
```

```
INTO public.admin_audit_logs (
```

```
action, entity_type, entity_id,
```

```
actor_id, actor_role, after_value
```

```
) VALUES (
```

```
'deactivate_user',
```

```
'profiles',
```

```
target_user_id::text,
```

```
auth.uid(),
```

```
actor_role::text,
```

```
jsonb_build_object(
```



```
'reason', reason,
```

```
'deactivated_at', now()
```

```
)
```

```
);
```

```
jsonb_build_object(
```

```
RETURN
```

```
'success', true,
```

```
'user_id', target_user_id,
```

```
'deactivated_at', now(),
```

```
'reason', reason
```

```
);
```

```
END;
```

```
$function$;
```

```
-- ----- hg_decrement_promo_used_count -----  
---
```

```
CREATE OR REPLACE FUNCTION  
public.hg_decrement_promo_used_count()  
  RETURNS trigger  
  LANGUAGE plpgsql  
  SECURITY DEFINER  
  SET search_path TO 'public'  
AS $function$  
BEGIN  
  UPDATE public.promo_codes  
  SET used_count = GREATEST(used_count - 1, 0)  
  WHERE id = OLD.promo_code_id;  
  RETURN OLD;  
END;  
$function$;
```

```

-- ----- hg_delete_storage_file -----
CREATE OR REPLACE FUNCTION
public.hg_delete_storage_file(bucket_name text,
file_path text)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$

                                DECLARE

project_url TEXT;

service_key TEXT;

storage_url TEXT;

                                BEGIN
                                IF
file_path IS NULL OR file_path = '' THEN

RETURN;

                                END IF;

-- Read project URL and service role key from
system_settings

-- These must be inserted into
system_settings (see STEP 6 below)

SELECT value::text INTO project_url

FROM public.system_settings

WHERE key = 'supabase_project_url';

SELECT value::text INTO service_key

```

```

        FROM public.system_settings

        WHERE key =
'supabase_service_role_key';

        IF project_url IS NULL OR
service_key IS NULL THEN

            RAISE WARNING
'hg_delete_storage_file: missing
supabase_project_url or supabase_service_role_key
in system_settings';

            RETURN;

            END IF;

            -- Remove
surrounding quotes if stored as JSON string

            project_url :=
trim(both '"' from project_url);

            service_key :=
trim(both '"' from service_key);

            storage_url
:= project_url || '/storage/v1/object/' ||
bucket_name || '/' || file_path;

            PERFORM

net.http_delete(

url      := storage_url,

```

```

headers := jsonb_build_object(

    'Authorization', 'Bearer ' || service_key,

    'Content-Type', 'application/json'

)

);

END;

$function$

-- ----- hg_enforce_single_default_address ---
-----
CREATE OR REPLACE FUNCTION
public.hg_enforce_single_default_address()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    IF NEW.is_default = TRUE THEN
        UPDATE public.user_addresses
        SET is_default = FALSE
        WHERE user_id = NEW.user_id AND id != NEW.id
    AND is_default = TRUE;
    END IF;
    RETURN NEW;
END;

```

```

$function$;

-- ----- hg_extract_storage_path -----
CREATE OR REPLACE FUNCTION
public.hg_extract_storage_path(url text)
RETURNS text
LANGUAGE plpgsql
SET search_path TO 'public'
AS $function$
DECLARE
    path TEXT;
BEGIN
    -- Handles full Supabase storage URLs
    -- Input:
    https://xxx.supabase.co/storage/v1/object/public/me
    nu-images/abc.jpg
    -- Output: abc.jpg
    IF url IS NULL OR url = '' THEN
        RETURN NULL;
    END IF;
    -- Extract everything after
    /object/public/{bucket}/
    path := regexp_replace(url,
'^.*\/object\/public\/[^\]+\/', '');
    IF path = url THEN
        -- URL format not
        recognised – return as-is
        RETURN url;
    END IF;
    RETURN path;
END;
$function$;

-- ----- hg_generate_referral_code -----
CREATE OR REPLACE FUNCTION
public.hg_generate_referral_code()
RETURNS text
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'

```



```

AS $function$
DECLARE
    v_code TEXT;
    v_exists BOOLEAN;
BEGIN
    LOOP
        v_code :=
upper(substring(encode(gen_random_bytes(6), 'hex'),
1, 8));
        SELECT EXISTS(SELECT 1 FROM public.profiles
WHERE referral_code = v_code) INTO v_exists;
        EXIT WHEN NOT v_exists;
    END LOOP;
    RETURN v_code;
END;
$function$;

-- ----- hg_generate_referral_code -----
CREATE OR REPLACE FUNCTION
public.hg_generate_referral_code(p_email text)
RETURNS text
LANGUAGE plpgsql
SET search_path TO 'public'
AS $function$
declare
    base text;
    candidate text;
begin
    base :=
upper(regex_replace(split_part(coalesce(p_email,
gen_random_uuid()::text), '@', 1), '^[a-zA-Z0-9]',
'', 'g'));
    if length(base) < 4 then
        base := 'HG' ||
upper(substr(replace(gen_random_uuid()::text, '-',
''), 1, 6));
    end if;

    loop
        candidate := substr(base, 1, 6) ||

```

```

upper(substr(replace(gen_random_uuid()::text, '-',
''), 1, 4));
        exit when not exists (select 1 from profiles
where referral_code = candidate);
    end loop;

    return candidate;
end;
$function$;

-- ----- hg_guard_access_code_assignment -----
-----
CREATE OR REPLACE FUNCTION
public.hg_guard_access_code_assignment()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    IF NEW.status = 'assigned' AND OLD.status !=
'available' THEN
        RAISE EXCEPTION 'Access code % is not available
(status: %)', OLD.code, OLD.status;
    END IF;
    RETURN NEW;
END;
$function$;

-- ----- hg_handle_new_auth_user -----
CREATE OR REPLACE FUNCTION
public.hg_handle_new_auth_user()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_full_name text;
    v_phone text;

```

```

begin
    v_full_name := coalesce(new.raw_user_meta_data->>'full_name', new.raw_user_meta_data->>'name');
    v_phone := new.raw_user_meta_data->>'phone';

    insert into public.profiles (id, email,
full_name, phone, referral_code)
    values (new.id, new.email, v_full_name, v_phone,
public.hg_generate_referral_code(new.email))
    on conflict (id) do update set
        email = excluded.email,
        full_name = coalesce(public.profiles.full_name,
excluded.full_name),
        phone = coalesce(public.profiles.phone,
excluded.phone),
        updated_at = now();

    insert into public.wallets (user_id, balance,
currency)
    values (new.id, 0, 'NGN')
    on conflict (user_id) do nothing;

    return new;
end;
$function$;

-- ----- hg_increment_promo_used_count -----
---
CREATE OR REPLACE FUNCTION
public.hg_increment_promo_used_count()
    RETURNS trigger
    LANGUAGE plpgsql
    SECURITY DEFINER
    SET search_path TO 'public'
AS $function$
BEGIN
    UPDATE public.promo_codes SET used_count =
used_count + 1 WHERE id = NEW.promo_code_id;
    RETURN NEW;
END;

```

```

$function$;

-- ----- hg_is_admin -----
CREATE OR REPLACE FUNCTION public.hg_is_admin()
RETURNS boolean
LANGUAGE sql
STABLE
SET search_path TO 'public'
AS $function$
    SELECT EXISTS (
        SELECT 1 FROM public.profiles
        WHERE id = (select auth.uid())
        AND role IN ('admin'::user_role,
'super_admin'::user_role, 'superadmin'::user_role)
    );
$function$;

-- ----- hg_log_order_status_change -----
CREATE OR REPLACE FUNCTION
public.hg_log_order_status_change()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    IF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO public.order_status_logs (order_id,
status, note)
        VALUES (NEW.id, NEW.status, 'Auto-logged by
trigger');
    END IF;
    RETURN NEW;
END;
$function$;

-- ----- hg_mark_order_paid -----
CREATE OR REPLACE FUNCTION
public.hg_mark_order_paid(p_order_id uuid,
p_provider text DEFAULT NULL::text,

```

```

p_provider_reference text DEFAULT NULL::text,
p_amount numeric DEFAULT NULL::numeric, p_metadata
jsonb DEFAULT '{} '::jsonb)
RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_order record;
    v_payment_id uuid;
begin
    select * into v_order
    from orders
    where id = p_order_id
    for update;

    if v_order.id is null then
        raise exception 'Order not found: %',
p_order_id;
    end if;

    if v_order.payment_status =
'paid'::payment_status then
        return null;
    end if;

    update orders
    set payment_status = 'paid'::payment_status,
        status = case when status =
'received'::order_status then 'paid'::order_status
else status end,
        paid_at = coalesce(paid_at, now()),
        payment_confirmed_at =
coalesce(payment_confirmed_at, now()),
        updated_at = now()
    where id = p_order_id;

    if p_provider is not null and
p_provider_reference is not null then

```

```

        insert into payments (order_id, user_id,
        provider, reference, amount, status, confirmed_at,
        metadata)
        values (p_order_id, v_order.user_id,
        p_provider, p_provider_reference,
        coalesce(p_amount, v_order.total_amount), 'paid',
        now(), coalesce(p_metadata, '{}'::jsonb))
        on conflict (reference) do update set
        status = 'paid',
        confirmed_at = excluded.confirmed_at,
        metadata = payments.metadata ||
        excluded.metadata
        returning id into v_payment_id;
    end if;

    return v_payment_id;
end;
$function$;

```

```

-- ----- hg_on_banner_delete -----
CREATE OR REPLACE FUNCTION
public.hg_on_banner_delete()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$

```

BEGIN

```

IF
OLD.image_url IS NOT NULL AND OLD.image_url <> ''
THEN

```

```
PERFORM hg_delete_storage_file(  
  
    'banners',  
  
    hg_extract_storage_path(OLD.image_url)  
  
    );  
  
END IF;  
  
    IF OLD.mobile_image_url IS NOT  
NULL AND OLD.mobile_image_url <> '' THEN  
  
        PERFORM  
hg_delete_storage_file(  
  
            'banners',
```

```

hg_extract_storage_path(OLD.mobile_image_url)

);

END IF;

RETURN

OLD;

END;

$function$;

-- ----- hg_on_event_delete -----
CREATE OR REPLACE FUNCTION
public.hg_on_event_delete()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'

```



```
AS $function$

BEGIN

IF OLD.image_url IS NOT NULL AND OLD.image_url <>
'' THEN

    PERFORM hg_delete_storage_file(

        'event-images',

        hg_extract_storage_path(OLD.image_url)

    );
```

```
END IF;
```

```
RETURN OLD;
```

```
END;
```

```
$function$;
```

```
-- ----- hg_on_hp_transaction_insert -----  
-
```

```
CREATE OR REPLACE FUNCTION  
public.hg_on_hp_transaction_insert()  
  RETURNS trigger  
  LANGUAGE plpgsql  
  SECURITY DEFINER  
  SET search_path TO 'public'  
AS $function$  
BEGIN  
  -- Update last_hp_activity_at for earn  
  transactions (idempotent max)  
  IF NEW.type = 'earn' THEN  
    UPDATE public.profiles  
    SET  
      last_hp_activity_at = GREATEST(  

```

```

        COALESCE(last_hp_activity_at,
NEW.created_at),
        NEW.created_at
    ),
    updated_at = now()
WHERE id = NEW.user_id
    AND (last_hp_activity_at IS NULL OR
last_hp_activity_at < NEW.created_at);
END IF;

-- Re-evaluate tier (idempotent – only writes if
tier changed)
PERFORM
public.hg_set_profile_tier_from_balance(NEW.user_id
);

RETURN NEW;
END;
$function$;

-- ----- hg_on_hp_transaction_reconcile -----
----
CREATE OR REPLACE FUNCTION
public.hg_on_hp_transaction_reconcile()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_user_id UUID;
    v_balance INTEGER;
BEGIN
    v_user_id := COALESCE(NEW.user_id, OLD.user_id);

    SELECT GREATEST(0, COALESCE(SUM(
        CASE WHEN type IN ('earn','adjustment') THEN
amount ELSE -amount END
    ), 0))::INTEGER
    INTO v_balance

```

```

FROM public.hp_transactions
WHERE user_id = v_user_id;

UPDATE public.profiles
SET
    hp_balance = v_balance,
    updated_at = now()
WHERE id = v_user_id
    AND hp_balance IS DISTINCT FROM v_balance;

PERFORM
public.hg_set_profile_tier_from_balance(v_user_id);

RETURN COALESCE(NEW, OLD);
END;
$function$;

-- ----- hg_on_marketplace_listing_delete -----
-----
CREATE OR REPLACE FUNCTION
public.hg_on_marketplace_listing_delete()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$

BEGIN

IF OLD.image_url IS NOT NULL AND

```

```
OLD.image_url <> '' THEN

    PERFORM hg_delete_storage_file(

        'marketplace-images',

        hg_extract_storage_path(OLD.image_url)

    );

END IF;
```

```
RETURN OLD;
```

```
END;
```

```
$function$;
```

```
-- ----- hg_on_menu_item_delete -----  
CREATE OR REPLACE FUNCTION  
public.hg_on_menu_item_delete()  
RETURNS trigger  
LANGUAGE plpgsql  
SECURITY DEFINER  
SET search_path TO 'public'  
AS $function$
```

```
BEGIN
```

```
IF OLD.image_url IS NOT NULL AND  
OLD.image_url <> '' THEN
```

```
PERFORM hg_delete_storage_file(
```

```

        'menu-images',

hg_extract_storage_path(OLD.image_url)

    );

END IF;

RETURN OLD;

END;

$function$

-- ----- hg_on_profile_delete -----
CREATE OR REPLACE FUNCTION
public.hg_on_profile_delete()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$

```

```
BEGIN
```

```
IF
```

```
OLD.photo_url IS NOT NULL AND OLD.photo_url <> ''  
THEN
```

```
PERFORM
```

```
hg_delete_storage_file(  
    'profile-photos',
```



```
hg_extract_storage_path(OLD.photo_url)
```

```
);
```

```
END IF;
```

```
RETURN OLD;
```

```
END;
```

```

                                $function$;

-- ----- hg_on_profile_photo_update -----
CREATE OR REPLACE FUNCTION
public.hg_on_profile_photo_update()
  RETURNS trigger
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
AS $function$
```

```

                                BEGIN
```

```

                                --
Only fire if photo_url actually changed and old one
existed
```

```

                                IF
```

```
OLD.photo_url IS NOT NULL
```

```
AND OLD.photo_url <> ''
```

```
AND OLD.photo_url IS DISTINCT FROM  
NEW.photo_url
```

```
THEN
```

```
PERFORM hg_delete_storage_file(
```

```
'profile-photos',
```

```
hg_extract_storage_path(OLD.photo_url)
```

```
);
```

```
END IF;
```

```
RETURN NEW;
```

END;

\$function\$;

```
-- ----- hg_on_reward_delete -----  
CREATE OR REPLACE FUNCTION  
public.hg_on_reward_delete()  
  RETURNS trigger  
  LANGUAGE plpgsql  
  SECURITY DEFINER  
  SET search_path TO 'public'  
AS $function$
```

BEGIN

```
                                IF OLD.image_url  
IS NOT NULL AND OLD.image_url <> '' THEN
```

PERFORM

```
hg_delete_storage_file(
```

```
images',                                'reward-

hg_extract_storage_path(OLD.image_url)

);

END IF;

RETURN OLD;
```

```
END;
```

```
$function$;
```

```
-- ----- hg_place_order -----  
CREATE OR REPLACE FUNCTION  
public.hg_place_order(p_user_id uuid, p_guest_name  
text, p_guest_email citext, p_guest_phone text,  
p_items jsonb, p_delivery_fee numeric,  
p_discount_amount numeric,  
p_delivery_address_snapshot jsonb,  
p_delivery_window_id uuid, p_notes text,  
p_scheduled_for timestamp with time zone,  
p_wallet_amount_used numeric, p_hp_redeemed  
integer, p_idempotency_key text DEFAULT NULL::text,  
p_promo_code_id uuid DEFAULT NULL::uuid)  
RETURNS uuid  
LANGUAGE plpgsql  
SECURITY DEFINER  
SET search_path TO 'public'  
AS $function$  
DECLARE  
    v_order_id uuid;  
    v_item jsonb;  
    v_menu_item record;  
    v_quantity integer;  
    v_options jsonb;  
    v_line_total numeric(14,2);  
    v_subtotal numeric(14,2) := 0;  
    v_total_amount numeric(14,2);  
    v_card_amount_used numeric(14,2);  
    v_hp_earned integer := 0;  
    v_now timestamptz := now();  
    v_accepting_orders text;
```

```

v_daily_capacity text;
v_active_orders_today integer;
v_window_capacity integer;
v_window_active_orders integer;
v_promo record;
v_user_uses integer;
v_item_ordered_today integer;
v_addon jsonb;
v_addon_record record;
v_order_item_id uuid;
v_addon_price_sum numeric(14,2);
BEGIN
    -- Object Ownership check (Student A cannot place
    order for Student B; Guests cannot place order for
    authenticated users)
    IF p_user_id IS NOT NULL THEN
        IF auth.uid() IS NULL THEN
            RAISE EXCEPTION 'hg_place_order: Guest
sessions cannot place orders for authenticated
accounts.';
        ELSIF p_user_id <> auth.uid() AND (SELECT role
FROM public.profiles WHERE id = auth.uid()) NOT IN
('admin'::user_role, 'super_admin'::user_role,
'superadmin'::user_role) THEN
            RAISE EXCEPTION 'hg_place_order: Unauthorized
to place order for another user.';
        END IF;
    END IF;

    -- 1. Idempotency Key Check
    IF p_idempotency_key IS NOT NULL THEN
        SELECT id INTO v_order_id FROM public.orders
WHERE idempotency_key = p_idempotency_key;
        IF FOUND THEN
            RETURN v_order_id;
        END IF;
    END IF;

    -- 2. Validate user vs guest input
    IF p_user_id is null and p_guest_email is null

```



```

and p_guest_phone is null then
    raise exception 'Guest checkout requires
guest_email or guest_phone';
END IF;

IF p_user_id is null and
(coalesce(p_wallet_amount_used, 0) > 0 or
coalesce(p_hp_redeemed, 0) > 0) then
    raise exception 'Guests cannot use wallet or HP
redemption';
END IF;

IF jsonb_typeof(p_items) <> 'array' or
jsonb_array_length(p_items) = 0 then
    raise exception 'Order requires at least one
item';
END IF;

-- 3. Verify Kitchen Capacity Settings
SELECT value INTO v_accepting_orders FROM
public.kitchen_settings WHERE key =
'accepting_orders';
IF v_accepting_orders = 'false' THEN
    RAISE EXCEPTION 'hg_place_order: Kitchen is
currently not accepting orders.';
END IF;

SELECT value INTO v_daily_capacity FROM
public.kitchen_settings WHERE key =
'daily_order_capacity';
IF v_daily_capacity IS NOT NULL AND
v_daily_capacity <> '' THEN
    SELECT COUNT(*) INTO v_active_orders_today
    FROM public.orders
    WHERE status <> 'cancelled' AND
created_at::date = CURRENT_DATE;

IF v_active_orders_today >=
v_daily_capacity::integer THEN
    RAISE EXCEPTION 'hg_place_order: Kitchen has

```

```

reached its daily order capacity limit.';
    END IF;
END IF;

-- 4. Verify Delivery Window Capacity
IF p_delivery_window_id IS NOT NULL THEN
    SELECT capacity INTO v_window_capacity
    FROM public.delivery_windows
    WHERE id = p_delivery_window_id FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'hg_place_order: Selected
delivery window not found.';
    END IF;

    -- Count active orders for this delivery window
    on the target date (or today if scheduled_for is
    null)
    SELECT COUNT(*) INTO v_window_active_orders
    FROM public.orders
    WHERE delivery_window_id = p_delivery_window_id
        AND status <> 'cancelled'
        AND COALESCE(scheduled_for::date,
created_at::date) = COALESCE(p_scheduled_for::date,
CURRENT_DATE);

    IF v_window_capacity IS NOT NULL AND
v_window_active_orders >= v_window_capacity THEN
        RAISE EXCEPTION 'hg_place_order: Selected
delivery window has reached its capacity limit.';
    END IF;
END IF;

-- Create stage table
CREATE temporary TABLE IF NOT EXISTS
pg_temp.hg_order_items_stage (
    id uuid DEFAULT gen_random_uuid(),
    menu_item_id uuid,
    name_snapshot text,
    price_snapshot numeric(14,2),

```

```

        hp_earn_snapshot integer,
        quantity integer,
        options_snapshot jsonb,
        line_total numeric(14,2),
        addons jsonb
    ) ON COMMIT DROP;

TRUNCATE TABLE pg_temp.hg_order_items_stage;

-- 5. Process Menu Items, Check Daily Limits &
Addons
FOR v_item IN SELECT * FROM
jsonb_array_elements(p_items)
LOOP
    v_quantity := coalesce((v_item->
>'quantity')::integer, 0);
    IF v_quantity <= 0 OR v_quantity > 50 THEN
        RAISE EXCEPTION 'Invalid item quantity';
    END IF;

    v_options := coalesce(v_item->'options',
'{}'::jsonb);

    -- Row-lock menu item
    SELECT id, name, price, hp_earn, is_available,
daily_limit
    INTO v_menu_item
    FROM menu_items
    WHERE id = (v_item->'menu_item_id')::uuid
    FOR SHARE;

    IF v_menu_item.id IS NULL THEN
        RAISE EXCEPTION 'Menu item not found: %',
v_item->'menu_item_id';
    END IF;

    IF NOT v_menu_item.is_available THEN
        RAISE EXCEPTION 'Menu item is not available:
%', v_menu_item.name;
    END IF;

```

```

-- Check Menu Item Daily Limits
IF v_menu_item.daily_limit IS NOT NULL THEN
    SELECT COALESCE(SUM(oi.quantity), 0) INTO
v_item_ordered_today
    FROM public.order_items oi
    JOIN public.orders o ON oi.order_id = o.id
    WHERE oi.menu_item_id = v_menu_item.id
    AND o.status <> 'cancelled'
    AND o.created_at::date = CURRENT_DATE;

    IF (v_item_ordered_today + v_quantity) >
v_menu_item.daily_limit THEN
        RAISE EXCEPTION 'hg_place_order: Daily
limit of % reached for menu item: % (Ordered: %,
Requested: %)',
            v_menu_item.daily_limit,
v_menu_item.name, v_item_ordered_today, v_quantity;
    END IF;
END IF;

-- Calculate selected addons sum
v_addon_price_sum := 0;
IF v_item->'addons' IS NOT NULL AND
jsonb_typeof(v_item->'addons') = 'array' THEN
    FOR v_addon IN SELECT * FROM
jsonb_array_elements(v_item->'addons')
    LOOP
        SELECT price INTO v_addon_record FROM
public.menu_addons WHERE id = (v_addon-
>>'addon_id')::uuid FOR SHARE;
        IF FOUND THEN
            v_addon_price_sum := v_addon_price_sum +
(coalesce(v_addon_record.price, 0) *
coalesce((v_addon->>'quantity')::integer, 1));
        END IF;
    END LOOP;
END IF;

v_line_total := (v_menu_item.price +

```

```

v_addon_price_sum) * v_quantity;
    v_subtotal := v_subtotal + v_line_total;

    IF p_user_id IS NOT NULL THEN
        v_hp_earned := v_hp_earned +
(coalesce(v_menu_item.hp_earn, 0) * v_quantity);
    END IF;

    INSERT INTO pg_temp.hg_order_items_stage (
        menu_item_id, name_snapshot, price_snapshot,
hp_earn_snapshot, quantity, options_snapshot,
line_total, addons
    ) VALUES (
        v_menu_item.id, v_menu_item.name,
v_menu_item.price, coalesce(v_menu_item.hp_earn,
0), v_quantity, v_options, v_line_total, v_item-
>'addons'
    );
END LOOP;

-- 6. Validate and Consume Promo Code
IF p_promo_code_id IS NOT NULL THEN
    SELECT * INTO v_promo FROM public.promo_codes
WHERE id = p_promo_code_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'hg_place_order: Promo code
not found.';
    END IF;

    IF NOT v_promo.is_active THEN
        RAISE EXCEPTION 'hg_place_order: Promo code
is inactive.';
    END IF;

    IF v_promo.starts_at > now() OR v_promo.ends_at
< now() THEN
        RAISE EXCEPTION 'hg_place_order: Promo code
has expired or is not yet active.';
    END IF;

```

```

        IF v_promo.max_uses IS NOT NULL AND
v_promo.used_count >= v_promo.max_uses THEN
            RAISE EXCEPTION 'hg_place_order: Promo code
usage limit has been fully reached.';
        END IF;

        IF v_subtotal <
COALESCE(v_promo.min_order_amount, 0) THEN
            RAISE EXCEPTION 'hg_place_order: Order
subtotal (%) does not meet the minimum required
order amount (%) for this promo code.',
                v_subtotal, v_promo.min_order_amount;
        END IF;

        IF p_user_id IS NOT NULL AND
v_promo.max_uses_per_user IS NOT NULL THEN
            SELECT COUNT(*) INTO v_user_uses
            FROM public.orders
            WHERE user_id = p_user_id AND promo_code_id =
p_promo_code_id AND status <> 'cancelled';

            IF v_user_uses >= v_promo.max_uses_per_user
THEN
                RAISE EXCEPTION 'hg_place_order: You have
reached your maximum usage limit for this promo
code.';
            END IF;
        END IF;

        -- Deduct promo use limit
        UPDATE public.promo_codes SET used_count =
used_count + 1 WHERE id = p_promo_code_id;
        END IF;

        v_total_amount := greatest(0, v_subtotal +
coalesce(p_delivery_fee, 0) -
coalesce(p_discount_amount, 0));

        IF coalesce(p_wallet_amount_used, 0) >
v_total_amount then

```

```

        raise exception 'wallet_amount_used cannot
exceed order total';
    END IF;

    v_card_amount_used := greatest(0, v_total_amount
- coalesce(p_wallet_amount_used, 0));

-- 7. Create Order Row
INSERT INTO public.orders (
    user_id,
    guest_name,
    guest_email,
    guest_phone,
    status,
    payment_status,
    subtotal,
    delivery_fee,
    discount_amount,
    total_amount,
    hp_earned,
    hp_redeemed,
    wallet_amount_used,
    card_amount_used,
    delivery_address_snapshot,
    delivery_window_id,
    notes,
    scheduled_for,
    paid_at,
    payment_confirmed_at,
    idempotency_key,
    promo_code_id
) VALUES (
    p_user_id,
    p_guest_name,
    p_guest_email,
    p_guest_phone,
    CASE WHEN p_user_id IS NOT NULL AND
v_total_amount > 0 AND
coalesce(p_wallet_amount_used, 0) = v_total_amount
THEN 'paid'::order_status ELSE

```

```

'received'::order_status END,
    CASE WHEN p_user_id IS NOT NULL AND
v_total_amount > 0 AND
coalesce(p_wallet_amount_used, 0) = v_total_amount
THEN 'paid'::payment_status ELSE
'pending'::payment_status END,
    v_subtotal,
    coalesce(p_delivery_fee, 0),
    coalesce(p_discount_amount, 0),
    v_total_amount,
    v_hp_earned,
    coalesce(p_hp_redeemed, 0),
    coalesce(p_wallet_amount_used, 0),
    v_card_amount_used,
    coalesce(p_delivery_address_snapshot,
'{}'::jsonb),
    p_delivery_window_id,
    p_notes,
    p_scheduled_for,
    CASE WHEN p_user_id IS NOT NULL AND
v_total_amount > 0 AND
coalesce(p_wallet_amount_used, 0) = v_total_amount
THEN v_now ELSE null END,
    CASE WHEN p_user_id IS NOT NULL AND
v_total_amount > 0 AND
coalesce(p_wallet_amount_used, 0) = v_total_amount
THEN v_now ELSE null END,
    p_idempotency_key,
    p_promo_code_id
) RETURNING id INTO v_order_id;

-- Log promo code use
IF p_promo_code_id IS NOT NULL THEN
    INSERT INTO public.promo_code_uses
(promo_code_id, user_id, order_id, discount_amount,
created_at)
    VALUES (p_promo_code_id, p_user_id, v_order_id,
coalesce(p_discount_amount, 0), v_now);
END IF;

```



```

-- 8. Save Order Items and Addon Selections
FOR v_item IN SELECT * FROM
pg_temp.hg_order_items_stage
LOOP
    INSERT INTO public.order_items (
        order_id, menu_item_id, name_snapshot,
        price_snapshot, hp_earn_snapshot, quantity,
        options_snapshot, line_total
    ) VALUES (
        v_order_id, v_item.menu_item_id,
        v_item.name_snapshot, v_item.price_snapshot,
        v_item.hp_earn_snapshot, v_item.quantity,
        v_item.options_snapshot, v_item.line_total
    ) RETURNING id INTO v_order_item_id;

    -- Process selected addons for this order item
    IF v_item.addons IS NOT NULL AND
    jsonb_typeof(v_item.addons) = 'array' THEN
        FOR v_addon IN SELECT * FROM
        jsonb_array_elements(v_item.addons)
        LOOP
            SELECT id, name, price, group_id INTO
            v_addon_record FROM public.menu_addons WHERE id =
            (v_addon->>'addon_id')::uuid FOR SHARE;
            IF FOUND THEN
                INSERT INTO public.order_addon_selections
                (
                    order_item_id, addon_id, group_id,
                    name_snapshot, price_delta_snapshot, quantity
                ) VALUES (
                    v_order_item_id, v_addon_record.id,
                    v_addon_record.group_id, v_addon_record.name,
                    v_addon_record.price, coalesce((v_addon-
                    >>'quantity')::integer, 1)
                );
            END IF;
        END LOOP;
    END IF;
END LOOP;

```

```

-- 9. Atomic Wallet/HP Debit where applicable
IF p_user_id IS NOT NULL AND
coalesce(p_hp_redeemed, 0) > 0 THEN
    PERFORM public.hg_add_hp_transaction(
        p_user_id,
        'spend',
        p_hp_redeemed,
        'order_redemption',
        'order',
        v_order_id,
        null,
        jsonb_build_object('source',
'atomic_order_processing'),
        null
    );
END IF;

IF p_user_id IS NOT NULL AND
coalesce(p_wallet_amount_used, 0) > 0 THEN
    PERFORM public.hg_add_wallet_transaction(
        p_user_id,
        'debit',
        p_wallet_amount_used,
        'Order payment',
        'order',
        v_order_id,
        null,
        null,
        null,
        jsonb_build_object('source',
'atomic_order_processing')
    );
END IF;

RETURN v_order_id;
END;
$function$;

-- ----- hg_process_referral_reward -----
CREATE OR REPLACE FUNCTION

```

```

public.hg_process_referral_reward(p_referred_user_id
d uuid, p_trigger_order_id uuid)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_referral referrals%rowtype;
    v_setting jsonb;
    v_ref_amount integer := 200;
    v_new_amount integer := 100;
begin
    select * into v_referral
    from referrals
    where referred_user_id = p_referred_user_id and
status = 'pending'
    limit 1;

    if not found then
        return;
    end if;

    select value into v_setting from system_settings
where key = 'referral_hp_reward';
    if found and v_setting is not null then
        v_ref_amount := coalesce((v_setting->>'amount')::integer, 200);
        v_new_amount := coalesce((v_setting->>'referred_amount')::integer, 100);
    end if;

    perform public.hg_add_hp_transaction(
        v_referral.referrer_id,
        'earn',
        v_ref_amount,
        'referral_reward',
        'referral',
        v_referral.id,
        null,

```

```

        jsonb_build_object('trigger_order_id',
p_trigger_order_id),
        now() + interval '12 months'
    );

perform public.hg_add_hp_transaction(
    p_referred_user_id,
    'earn',
    v_new_amount,
    'referral_welcome',
    'referral',
    v_referral.id,
    null,
    jsonb_build_object('trigger_order_id',
p_trigger_order_id),
    now() + interval '12 months'
);

update referrals
set status = 'rewarded',
    trigger_order_id = p_trigger_order_id,
    hp_awarded = v_ref_amount
where id = v_referral.id;
end;
$function$;

-- ----- hg_purchase_marketplace_item -----
--
CREATE OR REPLACE FUNCTION
public.hg_purchase_marketplace_item(p_user_id uuid,
p_listing_id uuid, p_quantity integer DEFAULT 1,
p_pay_with_hp boolean DEFAULT false,
p_wallet_amount numeric DEFAULT 0)
RETURNS marketplace_purchases
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_listing marketplace_listings%rowtype;

```

```

v_purchase marketplace_purchases%rowtype;
v_hp_cost integer := 0;
v_total_ngn numeric(14,2) := 0;
v_wallet_tx_id uuid;
v_hp_tx_id uuid;
v_code marketplace_access_codes%rowtype;
begin
    if p_quantity <= 0 then
        raise exception 'Quantity must be positive';
    end if;

    perform 1 from profiles where id = p_user_id for
update;
    if not found then
        raise exception 'User not found';
    end if;

    select * into v_listing
    from marketplace_listings
    where id = p_listing_id and status = 'active'
    for update;

    if not found then
        raise exception 'Listing not found or
inactive';
    end if;

    if v_listing.is_out_of_stock then
        raise exception 'Listing is out of stock';
    end if;

    if v_listing.inventory_count is not null and
v_listing.inventory_count < p_quantity then
        raise exception 'Insufficient inventory';
    end if;

    v_total_ngn := coalesce(v_listing.price, 0) *
p_quantity;
    v_hp_cost := coalesce(v_listing.hp_price, 0) *
p_quantity;

```

```

    if p_pay_with_hp then
        if v_listing.hp_price is null then
            raise exception 'Listing does not support HP
payment';
        end if;

        v_hp_tx_id := public.hg_add_hp_transaction(
            p_user_id,
            'spend',
            v_hp_cost,
            'marketplace_purchase',
            'marketplace_listing',
            p_listing_id,
            null,
            jsonb_build_object('quantity', p_quantity),
            null
        );
    elsif coalesce(p_wallet_amount, 0) > 0 then
        if p_wallet_amount < v_total_ngn then
            raise exception 'Wallet amount is less than
listing total';
        end if;

        v_wallet_tx_id :=
public.hg_add_wallet_transaction(
            p_user_id,
            'debit',
            p_wallet_amount,
            'Marketplace purchase',
            'marketplace_listing',
            p_listing_id,
            null,
            null,
            null,
            jsonb_build_object('quantity', p_quantity)
        );
    end if;

    if v_listing.inventory_count is not null then

```

```

        update marketplace_listings
        set inventory_count = inventory_count -
p_quantity
        where id = p_listing_id;
    end if;

    insert into marketplace_purchases (
        user_id, listing_id, quantity, pay_with_hp,
status, is_fulfilled,
        payment_method, wallet_amount, card_amount,
wallet_tx_id, hp_tx_id
    ) values (
        p_user_id,
        p_listing_id,
        p_quantity,
        p_pay_with_hp,
        'pending',
        false,
        case when p_pay_with_hp then 'hp' when
coalesce(p_wallet_amount, 0) > 0 then 'wallet' else
'card' end,
        case when p_pay_with_hp then 0 else
coalesce(p_wallet_amount, 0) end,
        case when p_pay_with_hp or
coalesce(p_wallet_amount, 0) > 0 then 0 else
v_total_ngn end,
        v_wallet_tx_id,
        v_hp_tx_id
    ) returning * into v_purchase;

    if v_listing.listing_type in
('code', 'digital_code', 'voucher', 'subscription')
then
        select * into v_code
        from marketplace_access_codes
        where listing_id = p_listing_id and status =
'available'
        limit 1 for update skip locked;

        if found then

```

```

        update marketplace_access_codes
        set status = 'assigned', assigned_purchase_id
= v_purchase.id, assigned_at = now()
        where id = v_code.id;
    end if;
end if;

    return v_purchase;
end;
$function$;

-- ----- hg_reactivate_user -----
CREATE OR REPLACE FUNCTION
public.hg_reactivate_user(target_user_id uuid)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$

```

DECLARE


```
actor_role user_role;
```

```
BEGIN
```

```
SELECT role INTO actor_role
```

```
FROM public.profiles
```

```
WHERE id = auth.uid();
```

```
IF actor_role <>  
'super_admin' THEN
```

```
                                RETURN
jsonb_build_object(

                                'success', false,

                                'error',
                                'Only super_admin can reactivate users'
```

);

IF;

END

UPDATE public.profiles SET

```
is_active      = true,
```

```
deactivated_at = NULL,
```

```
deactivated_by = NULL,
```

```
deactivation_reason = NULL,
```

```
updated_at          = now()
```

```
WHERE id = target_user_id;
```

```
INSERT INTO  
public.admin_audit_logs (
```

```
action, entity_type,  
entity_id, actor_id, actor_role
```

```
) VALUES (
```

```
'profiles', 'reactivate_user',
```

```
target_user_id::text, auth.uid(), actor_role::text
```



```
);
```

```
RETURN
```

```
jsonb_build_object(
```

```
'success', true,
```

```
'user_id', target_user_id,
```

```
'reactivated_at', now())
```

```
);
```

```
END;
```

```
$function$;
```

```
-- ----- hg_redeem_reward -----  
CREATE OR REPLACE FUNCTION  
public.hg_redeem_reward(p_user_id uuid, p_reward_id  
uuid)  
RETURNS reward_redemptions
```

```

LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_reward      public.rewards%ROWTYPE;
    v_profile      public.profiles%ROWTYPE;
    v_tier         public.hp_tiers%ROWTYPE;
    v_redemption   public.reward_redemptions%ROWTYPE;
    v_count        INTEGER;
    v_hp_cost      INTEGER;
    v_is_flash     BOOLEAN := FALSE;
BEGIN
    -- Row-lock profile
    SELECT * INTO v_profile FROM public.profiles
    WHERE id = p_user_id FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION
    'hg_redeem_reward: user not found'; END IF;

    -- Row-lock reward
    SELECT * INTO v_reward FROM public.rewards WHERE
    id = p_reward_id FOR UPDATE;
    IF NOT FOUND OR v_reward.is_active = FALSE THEN
        RAISE EXCEPTION 'hg_redeem_reward: reward not
    found or inactive';
    END IF;

    -- Determine if flash sale is active and valid
    IF coalesce(v_reward.flash_enabled, FALSE) AND
    v_reward.flash_starts_at <= NOW() AND
    v_reward.flash_ends_at >= NOW() THEN
        v_is_flash := TRUE;
        v_hp_cost := v_reward.flash_hp_cost;
        IF v_reward.flash_slots_remaining <= 0 THEN
            RAISE EXCEPTION 'hg_redeem_reward: flash
    slots are fully redeemed';
        END IF;
    ELSE
        v_hp_cost := v_reward.hp_cost;
        IF v_reward.expires_at IS NOT NULL AND

```

```

v_reward.expires_at < NOW() THEN
    RAISE EXCEPTION 'hg_redeem_reward: reward has
expired';
    END IF;
    IF v_reward.stock_quantity IS NOT NULL AND
v_reward.stock_quantity <= 0 THEN
        RAISE EXCEPTION 'hg_redeem_reward: reward is
out of stock';
    END IF;
    END IF;

    IF v_profile.hp_balance < v_hp_cost THEN
        RAISE EXCEPTION 'hg_redeem_reward: insufficient
HP. Balance: %, Cost: %',
            v_profile.hp_balance, v_hp_cost;
    END IF;

    -- Check per-user limit
    SELECT COUNT(*) INTO v_count
    FROM public.reward_redemptions
    WHERE user_id = p_user_id AND reward_id =
p_reward_id AND status != 'cancelled';
    IF v_count >= COALESCE(v_reward.max_per_user, 1)
    THEN
        RAISE EXCEPTION 'hg_redeem_reward: max
redemptions per user reached for this reward';
    END IF;

    -- Check tier requirement
    IF v_reward.min_tier_id IS NOT NULL THEN
        SELECT * INTO v_tier FROM public.hp_tiers WHERE
id = v_reward.min_tier_id;
        IF v_profile.current_tier_id IS NULL THEN
            RAISE EXCEPTION 'hg_redeem_reward: tier
requirement not met';
        END IF;
        IF (SELECT sort_order FROM public.hp_tiers
WHERE id = v_profile.current_tier_id) <
            v_tier.sort_order THEN
            RAISE EXCEPTION 'hg_redeem_reward: tier

```

```

requirement not met. Need: %', v_tier.name;
    END IF;
END IF;

-- Deduct HP
PERFORM public.hg_add_hp_transaction(
    p_user_id, 'redeem', -v_hp_cost,
    'reward_redemption', 'reward', p_reward_id,
    '{}', NULL, NULL
);

-- Decrement stock / flash slots
IF v_is_flash THEN
    UPDATE public.rewards SET flash_slots_remaining
= flash_slots_remaining - 1 WHERE id = p_reward_id;
ELSE
    IF v_reward.stock_quantity IS NOT NULL THEN
        UPDATE public.rewards SET stock_quantity =
stock_quantity - 1 WHERE id = p_reward_id;
    END IF;
END IF;

INSERT INTO public.reward_redemptions (user_id,
reward_id, status, hp_cost_snapshot)
VALUES (p_user_id, p_reward_id, 'pending',
v_hp_cost)
RETURNING * INTO v_redemption;

RETURN v_redemption;
END;
$function$;

-- ----- hg_run_hp_expiry_check -----
CREATE OR REPLACE FUNCTION
public.hg_run_hp_expiry_check(p_expiry_days integer
DEFAULT 180)
RETURNS integer
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'

```

```

AS $function$
declare
    v_record record;
    v_expired integer := 0;
    v_transaction_id uuid;
    v_previous_balance integer;
begin
    for v_record in
        select ht.id, ht.user_id,
        coalesce(ht.remaining_amount, ht.amount) as
        remaining_amount
        from hp_transactions ht
        where ht.type = 'earn'
        and coalesce(ht.remaining_amount, ht.amount)
        > 0
        and ht.created_at <= now() -
        make_interval(days => p_expiry_days)
        order by ht.created_at
        for update skip locked
    loop
        select hp_balance into v_previous_balance
        from profiles
        where id = v_record.user_id;

        v_transaction_id :=
        public.hg_add_hp_transaction(
            v_record.user_id,
            'expire',
            v_record.remaining_amount,
            'hp_expiry',
            'hp_transaction',
            v_record.id,
            null,
            jsonb_build_object('expired_by_job', true,
            'source_hp_transaction_id', v_record.id),
            null
        );

        update hp_transactions
        set remaining_amount = 0

```

```

        where id = v_record.id;

        insert into hp_expiry_log (
            user_id,
            hp_transaction_id,
            expired_amount,
            previous_balance,
            reason
        ) values (
            v_record.user_id,
            v_transaction_id,
            v_record.remaining_amount,
            coalesce(v_previous_balance, 0),
            'scheduled_expiry'
        );

        v_expired := v_expired + 1;
    end loop;

    return v_expired;
end;
$function$;

-- ----- hg_run_tier_maintenance_check -----
---
CREATE OR REPLACE FUNCTION
public.hg_run_tier_maintenance_check(p_user_id uuid
DEFAULT NULL::uuid)
RETURNS integer
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_profile    public.profiles%ROWTYPE;
    v_new_tier   public.hp_tiers%ROWTYPE;
    v_old_tier   public.hp_tiers%ROWTYPE;
    v_count      INTEGER := 0;
BEGIN
    FOR v_profile IN

```



```

SELECT * FROM public.profiles
WHERE is_active = TRUE
      AND (p_user_id IS NULL OR id = p_user_id)
FOR UPDATE SKIP LOCKED
LOOP
  SELECT * INTO v_new_tier FROM public.hp_tiers
  WHERE min_points <= v_profile.hp_balance
        AND is_active = TRUE
  ORDER BY min_points DESC LIMIT 1;

  IF NOT FOUND THEN CONTINUE; END IF;

  IF v_profile.current_tier_id IS DISTINCT FROM
v_new_tier.id THEN
    IF v_profile.current_tier_id IS NOT NULL THEN
      SELECT * INTO v_old_tier FROM
public.hp_tiers WHERE id =
v_profile.current_tier_id;
    END IF;

    INSERT INTO public.user_tiers (user_id,
tier_id, event, hp_at_event, previous_tier_id)
VALUES (
  v_profile.id, v_new_tier.id,
  CASE
    WHEN v_old_tier.sort_order IS NULL THEN
'upgrade'
    WHEN v_new_tier.sort_order >
v_old_tier.sort_order THEN 'upgrade'
    WHEN v_new_tier.sort_order <
v_old_tier.sort_order THEN 'downgrade'
    ELSE 'maintain'
  END,
  v_profile.hp_balance,
  v_profile.current_tier_id
);

  UPDATE public.profiles
  SET current_tier_id = v_new_tier.id,
updated_at = NOW()

```

```

        WHERE id = v_profile.id;

        v_count := v_count + 1;
    END IF;
END LOOP;
RETURN v_count;
END;
$function$;

-- ----- hg_set_profile_tier_from_balance -----
-----
CREATE OR REPLACE FUNCTION
public.hg_set_profile_tier_from_balance(p_user_id
uuid)
RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_tier_id uuid;
begin
    select id into v_tier_id
    from hp_tiers
    where min_points <= coalesce((select hp_balance
from profiles where id = p_user_id), 0)
    and coalesce(is_active, true) = true
    order by min_points desc, sort_order desc
    limit 1;

    if v_tier_id is not null then
        update profiles
        set current_tier_id = v_tier_id,
            updated_at = now()
        where id = p_user_id
        and current_tier_id is distinct from
v_tier_id;
    end if;

    return v_tier_id;

```

```

end;
$function$;

-- ----- hg_set_updated_at -----
CREATE OR REPLACE FUNCTION
public.hg_set_updated_at()
RETURNS trigger
LANGUAGE plpgsql
SET search_path TO 'public'
AS $function$
BEGIN NEW.updated_at := NOW(); RETURN NEW; END;
$function$;

-- ----- hg_sync_delivery_batch_window -----
---
CREATE OR REPLACE FUNCTION
public.hg_sync_delivery_batch_window()
RETURNS trigger
LANGUAGE plpgsql
SET search_path TO 'public'
AS $function$
BEGIN
    IF TG_OP = 'INSERT' THEN
        IF NEW.window_id IS NULL AND
NEW.delivery_window_id IS NOT NULL THEN
            NEW.window_id := NEW.delivery_window_id;
        ELSIF NEW.delivery_window_id IS NULL AND
NEW.window_id IS NOT NULL THEN
            NEW.delivery_window_id := NEW.window_id;
        END IF;
    ELSIF TG_OP = 'UPDATE' THEN
        -- whichever changed wins; if both changed,
prefer delivery_window_id
        IF NEW.delivery_window_id IS DISTINCT FROM
OLD.delivery_window_id THEN
            NEW.window_id := NEW.delivery_window_id;
        ELSIF NEW.window_id IS DISTINCT FROM
OLD.window_id THEN
            NEW.delivery_window_id := NEW.window_id;
        END IF;
    END IF;
END;

```

```

END IF;
RETURN NEW;
END;
$function$;

-- ----- hg_sync_hp_balance_from_ledger -----
----
CREATE OR REPLACE FUNCTION
public.hg_sync_hp_balance_from_ledger(p_user_id
uuid)
RETURNS integer
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_balance integer;
begin
    select greatest(0, coalesce(sum(
        case when type in ('earn','adjustment') then
amount else -amount end
    ), 0))::integer into v_balance
    from hp_transactions
    where user_id = p_user_id;

    update profiles
    set hp_balance = v_balance,
        last_hp_activity_at = coalesce((
        select max(created_at) from hp_transactions
where user_id = p_user_id and type = 'earn'
    ), last_hp_activity_at),
        updated_at = now()
    where id = p_user_id;

    perform
public.hg_set_profile_tier_from_balance(p_user_id);
    return v_balance;
end;
$function$;

```

```

-- ----- hg_sync_profile_wallet_to_wallets ---
-----
CREATE OR REPLACE FUNCTION
public.hg_sync_profile_wallet_to_wallets()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    IF NEW.wallet_balance IS DISTINCT FROM
    OLD.wallet_balance THEN
        INSERT INTO public.wallets (user_id, balance,
        currency, created_at, updated_at)
        VALUES (NEW.id, NEW.wallet_balance, 'NGN',
        NOW(), NOW())
        ON CONFLICT (user_id) DO UPDATE
        SET balance = EXCLUDED.balance, updated_at =
        NOW()
        WHERE wallets.balance IS DISTINCT FROM
        EXCLUDED.balance;
    END IF;
    RETURN NEW;
END;
$function$;

-- ----- hg_sync_referral_code_table -----
-
CREATE OR REPLACE FUNCTION
public.hg_sync_referral_code_table()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    IF NEW.referral_code IS NOT NULL THEN
        INSERT INTO public.referral_codes (user_id,
        code)
        VALUES (NEW.id, NEW.referral_code)

```

```

        ON CONFLICT DO NOTHING;
    END IF;
    RETURN NEW;
END;
$function$;

-- ----- hg_sync_review_hp_columns -----
CREATE OR REPLACE FUNCTION
public.hg_sync_review_hp_columns()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_hp_to_award INTEGER;
    v_exists BOOLEAN;
    v_order_user UUID;
BEGIN
    -- 1. Ensure columns are synchronized (priority
    to hp_awarded, fallback to hp_rewarded, default 20)
    IF TG_OP = 'INSERT' THEN
        IF NEW.hp_awarded IS NOT NULL THEN
            NEW.hp_rewarded := NEW.hp_awarded;
        ELSIF NEW.hp_rewarded IS NOT NULL THEN
            NEW.hp_awarded := NEW.hp_rewarded;
        ELSE
            -- Fetch standard review_hp from system
            settings
            SELECT coalesce((value->>'amount')::integer,
20) INTO v_hp_to_award
            FROM public.system_settings
            WHERE key = 'review_hp';

            IF v_hp_to_award IS NULL THEN
                v_hp_to_award := 20;
            END IF;

            NEW.hp_awarded := v_hp_to_award;
            NEW.hp_rewarded := v_hp_to_award;

```

```

END IF;

-- 2. Verify that the order actually belongs to
the user and is delivered
SELECT user_id INTO v_order_user
FROM public.orders
WHERE id = NEW.order_id AND status =
'delivered'::order_status;

IF v_order_user IS NULL OR v_order_user <>
NEW.user_id THEN
    RAISE EXCEPTION 'hg_sync_review_hp_columns:
Reviews can only be submitted for delivered orders
owned by the user.';
END IF;

-- 3. Enforce database-level uniqueness for HP
reward on this review
SELECT EXISTS (
    SELECT 1 FROM public.hp_transactions
    WHERE user_id = NEW.user_id
        AND reference_type = 'order_review'
        AND reference_id = NEW.order_id
) INTO v_exists;

IF v_exists THEN
    RAISE EXCEPTION 'hg_sync_review_hp_columns:
An HP transaction has already been issued for
reviewing this order %', NEW.order_id;
END IF;

-- 4. Issue HP transaction atomically through
the ledger
PERFORM public.hg_add_hp_transaction(
    NEW.user_id,
    'earn',
    NEW.hp_awarded,
    'order_review',
    'order_review',
    NEW.order_id,

```

```

        NULL,
        jsonb_build_object('order_id', NEW.order_id),
        now() + interval '12 months'
    );

    ELSE -- UPDATE (No manual change to reward
columns allowed)
        IF NEW.hp_awarded IS DISTINCT FROM
OLD.hp_awarded OR NEW.hp_rewarded IS DISTINCT FROM
OLD.hp_rewarded THEN
            RAISE EXCEPTION 'hg_sync_review_hp_columns:
Completed review HP columns cannot be modified.';
        END IF;
    END IF;

    RETURN NEW;
END;
$function$;

-- ----- hg_sync_wallet_balance_to_profile ---
-----
CREATE OR REPLACE FUNCTION
public.hg_sync_wallet_balance_to_profile()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    UPDATE public.profiles SET wallet_balance =
NEW.balance, updated_at = NOW()
    WHERE id = NEW.user_id;
    RETURN NEW;
END;
$function$;

-- ----- hg_trigger_referral_on_first_payment
-----
CREATE OR REPLACE FUNCTION
public.hg_trigger_referral_on_first_payment()

```



```

RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE v_order_count INTEGER;
BEGIN
    IF NEW.payment_status = 'paid' AND
    OLD.payment_status != 'paid'
        AND NEW.user_id IS NOT NULL THEN
        SELECT COUNT(*) INTO v_order_count
        FROM public.orders
        WHERE user_id = NEW.user_id AND payment_status
        = 'paid' AND id != NEW.id;
        IF v_order_count = 0 THEN
            PERFORM
            public.hg_process_referral_reward(NEW.user_id,
            NEW.id);
        END IF;
    END IF;
    RETURN NEW;
END;
$function$;

-- ----- hg_update_listing_stock_flag -----
--
CREATE OR REPLACE FUNCTION
public.hg_update_listing_stock_flag()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
begin
    if new.inventory_count is not null and
new.inventory_count <= 0 then
        new.is_out_of_stock := true;
        if new.status = 'active' then
            new.status := 'paused';
        end if;
    end if;
end;

```

```

        elsif new.inventory_count is not null and
new.inventory_count > 0 then
            new.is_out_of_stock := false;
        end if;
        new.updated_at := now();
        return new;
    end;
$function$;

-- ----- hg_verify_graduation_claim -----
CREATE OR REPLACE FUNCTION
public.hg_verify_graduation_claim()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    IF OLD.graduation_claimed IS TRUE AND
NEW.graduation_claimed IS FALSE THEN
        RAISE EXCEPTION 'hg_verify_graduation_claim:
Graduation claim status cannot be reverted.';
    END IF;

    RETURN NEW;
END;
$function$;

-- ----- hg_verify_order_status_transition ---
-----
CREATE OR REPLACE FUNCTION
public.hg_verify_order_status_transition()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    -- If inserting a new record, restrict starting
status

```

```

    IF TG_OP = 'INSERT' THEN
        IF NEW.status NOT IN ('received', 'paid') THEN
            RAISE EXCEPTION
            'hg_verify_order_status_transition: Initial order
            status must be received or paid.';
        END IF;
        RETURN NEW;
    END IF;

    -- If status is not changed, allow it
    IF OLD.status = NEW.status THEN
        RETURN NEW;
    END IF;

    -- Prevent transitioning out of terminal states
    IF OLD.status = 'cancelled' THEN
        RAISE EXCEPTION
        'hg_verify_order_status_transition: Cannot
        transition out of cancelled status.';
    END IF;

    IF OLD.status = 'refunded' THEN
        RAISE EXCEPTION
        'hg_verify_order_status_transition: Cannot
        transition out of refunded status.';
    END IF;

    IF OLD.status = 'delivered' AND NEW.status NOT IN
    ('refunded', 'unclaimed') THEN
        RAISE EXCEPTION
        'hg_verify_order_status_transition: Delivered order
        can only transition to refunded or unclaimed.';
    END IF;

    -- Strict forward validation transitions
    IF NEW.status = 'paid' AND OLD.status NOT IN
    ('received') THEN
        RAISE EXCEPTION
        'hg_verify_order_status_transition: Illegal
        transition to paid from %.', OLD.status;

```

```

END IF;

IF NEW.status = 'preparing' AND OLD.status NOT IN
('paid', 'received') THEN
    RAISE EXCEPTION
'hg_verify_order_status_transition: Illegal
transition to preparing from %.', OLD.status;
END IF;

IF NEW.status = 'ready' AND OLD.status NOT IN
('preparing') THEN
    RAISE EXCEPTION
'hg_verify_order_status_transition: Illegal
transition to ready from %.', OLD.status;
END IF;

RETURN NEW;
END;
$function$;

-- ----- hg_verify_promo_use_limit -----
CREATE OR REPLACE FUNCTION
public.hg_verify_promo_use_limit()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_max_per_user INTEGER;
    v_user_uses_count INTEGER;
BEGIN
    -- Get max_uses_per_user
    SELECT max_uses_per_user INTO v_max_per_user
    FROM public.promo_codes
    WHERE id = NEW.promo_code_id;

    IF v_max_per_user IS NOT NULL AND NEW.user_id IS
    NOT NULL THEN
        -- Count uses by this user

```

```

        SELECT COUNT(*) INTO v_user_uses_count
        FROM public.promo_code_uses
        WHERE promo_code_id = NEW.promo_code_id AND
        user_id = NEW.user_id AND id <> NEW.id;

        IF v_user_uses_count >= v_max_per_user THEN
            RAISE EXCEPTION 'hg_verify_promo_use_limit:
You have reached the maximum usage limit for this
promo code.';
        END IF;
    END IF;

    RETURN NEW;
END;
$function$;

-- ----- hg_verify_referral_integrity -----
--
CREATE OR REPLACE FUNCTION
public.hg_verify_referral_integrity()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    -- 1. Prevent self-referral
    IF NEW.referrer_id = NEW.referred_user_id THEN
        RAISE EXCEPTION 'hg_verify_referral_integrity:
Self-referral is strictly forbidden.';
    END IF;

    -- 2. Prevent circular referrals (A refers B and
    B refers A)
    IF EXISTS (
        SELECT 1 FROM public.referrals
        WHERE referrer_id = NEW.referred_user_id AND
        referred_user_id = NEW.referrer_id
    ) THEN
        RAISE EXCEPTION 'hg_verify_referral_integrity:

```

```

Circular referral chain detected and blocked.';
    END IF;

    RETURN NEW;
END;
$function$;

-- ----- hg_verify_refund_conservation -----
---
CREATE OR REPLACE FUNCTION
public.hg_verify_refund_conservation()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_order_total NUMERIC;
    v_total_refunded NUMERIC;
BEGIN
    IF NEW.type = 'refund' AND NEW.reference_type =
'order' AND NEW.reference_id IS NOT NULL THEN
        -- Get order total amount paid
        SELECT total_amount INTO v_order_total
        FROM public.orders
        WHERE id = NEW.reference_id;

        IF NOT FOUND THEN
            RAISE EXCEPTION
'hg_verify_refund_conservation: Referenced order %
not found.', NEW.reference_id;
        END IF;

        -- Calculate total refunded so far (including
current transaction if updated)
        SELECT COALESCE(SUM(amount), 0) INTO
v_total_refunded
        FROM public.wallet_transactions
        WHERE reference_type = 'order' AND reference_id
= NEW.reference_id AND type = 'refund' AND id <>

```

```

NEW.id;

    IF (v_total_refunded + NEW.amount) >
v_order_total THEN
        RAISE EXCEPTION
'hg_verify_refund_conservation: Total refunds (%)
cannot exceed original order amount (%).',
(v_total_refunded + NEW.amount), v_order_total;
    END IF;
END IF;

    RETURN NEW;
END;
$function$;

-- ----- hg_verify_rider_delivery_assignment -
-----
CREATE OR REPLACE FUNCTION
public.hg_verify_rider_delivery_assignment()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    -- If the order is updated to out_for_delivery or
delivered by a user with role rider:
    IF NEW.status IN ('out_for_delivery',
'delivered', 'delivery_attempted') AND
hg_current_user_role() = 'rider' THEN
        IF NOT EXISTS (
            SELECT 1 FROM public.delivery_assignments
            WHERE order_id = NEW.id AND rider_id =
auth.uid()
        ) THEN
            RAISE EXCEPTION
'hg_verify_rider_delivery_assignment: Riders can
only update orders assigned to them.';
        END IF;
    END IF;
END IF;

```

```

    RETURN NEW;
END;
$function$;

-- -----
hg_verify_wallet_conservation_invariant -----
CREATE OR REPLACE FUNCTION
public.hg_verify_wallet_conservation_invariant()
RETURNS trigger
LANGUAGE plpgsql
SET search_path TO 'public'
AS $function$
DECLARE
    v_ledger_sum NUMERIC;
BEGIN
    -- Sum of credits (credit, refund, adjustment)
    minus debits (debit)
    SELECT COALESCE(SUM(CASE WHEN type IN ('credit',
'refund') THEN amount WHEN type = 'debit' THEN -
amount ELSE 0 END), 0)
    INTO v_ledger_sum
    FROM public.wallet_transactions
    WHERE user_id = NEW.user_id;

    -- Add adjustments to the ledger sum
    SELECT v_ledger_sum + COALESCE(SUM(CASE WHEN type
= 'adjustment' THEN amount ELSE 0 END), 0)
    INTO v_ledger_sum
    FROM public.wallet_transactions
    WHERE user_id = NEW.user_id AND type =
'adjustment';

    -- We can log if there is any drift or enforce
strict check.
    -- Since some older records might be unledgered
during migration, we can enforce it for any active
transactional user.
    RETURN NEW;
END;

```



```

$function$;

-- ----- record_hp_transaction_atomic -----
--
CREATE OR REPLACE FUNCTION
public.record_hp_transaction_atomic(p_user_id uuid,
p_amount integer, p_type text, p_status text,
p_source text, p_reference_type text,
p_reference_id uuid, p_issued_by_admin_id uuid,
p_notes text)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_active_hp INTEGER;
    v_balance_after INTEGER;
    v_txn_id UUID;
BEGIN
    -- Idempotency check:
    IF p_reference_type IS NOT NULL AND
p_reference_id IS NOT NULL THEN
        SELECT id, balance_after INTO v_txn_id,
v_balance_after
        FROM public.hp_transactions
        WHERE user_id = p_user_id AND reference_type =
p_reference_type AND reference_id = p_reference_id
AND type = p_type;
        IF FOUND THEN
            RETURN jsonb_build_object('success', true,
'new_balance', v_balance_after, 'transaction_id',
v_txn_id, 'replayed', true);
        END IF;
    END IF;

    -- Row-lock profiles
    SELECT hp_balance INTO v_active_hp FROM
public.profiles WHERE id = p_user_id FOR UPDATE;
    IF NOT FOUND THEN

```

```

        RETURN jsonb_build_object('error', 'User
profile not found');
    END IF;

    IF p_status = 'pending' THEN
        v_balance_after := v_active_hp;
    ELSE
        IF p_type = 'spend' OR (p_amount < 0) THEN
            IF v_active_hp < abs(p_amount) THEN
                RETURN jsonb_build_object('error',
'Insufficient active HP balance');
            END IF;
            v_balance_after := v_active_hp -
abs(p_amount);
        ELSE
            v_balance_after := v_active_hp +
abs(p_amount);
        END IF;
    END IF;

    -- Insert ledger transaction
    INSERT INTO public.hp_transactions (
        user_id, amount, type, status, balance_after,
source, reference_type, reference_id,
issued_by_admin_id, metadata
    )
    VALUES (
        p_user_id, abs(p_amount), p_type, p_status,
v_balance_after, p_source, p_reference_type,
p_reference_id, p_issued_by_admin_id,
jsonb_build_object('notes', p_notes)
    )
    RETURNING id INTO v_txn_id;

    -- Update profiles hp_balance
    IF p_status <> 'pending' THEN
        UPDATE public.profiles SET hp_balance =
v_balance_after WHERE id = p_user_id;
    END IF;

```

```

        RETURN jsonb_build_object('success', true,
        'new_balance', v_balance_after, 'transaction_id',
        v_txn_id);
    END;
    $function$;

-- ----- register_for_event_atomic -----
CREATE OR REPLACE FUNCTION
public.register_for_event_atomic(p_event_id uuid,
p_user_id uuid)
    RETURNS jsonb
    LANGUAGE plpgsql
    SECURITY DEFINER
    SET search_path TO 'public'
AS $function$
    DECLARE
        v_capacity INTEGER;
        v_issued   INTEGER;
        v_ticket_id UUID;
        v_existing  UUID;
        v_status    TEXT;
    BEGIN
        SELECT capacity INTO v_capacity
        FROM   events
        WHERE  id = p_event_id
        FOR UPDATE;

        IF NOT FOUND THEN
            RETURN
            jsonb_build_object('error', 'Event not found');
        END IF;

        SELECT id, status
        INTO v_existing, v_status
        FROM
        event_tickets
        WHERE
        event_id = p_event_id
        AND
        user_id  = p_user_id
    
```

```

LIMIT
1;

IF
FOUND THEN

RETURN jsonb_build_object(

    'ticket_id',      v_existing,

    'status',         v_status,

    'already_registered', TRUE

);

END IF;

IF v_capacity IS NOT NULL
THEN

    SELECT COUNT(*) INTO
    v_issued FROM event_tickets WHERE event_id =
    p_event_id;

    IF v_issued >=
    v_capacity THEN

        RETURN
        jsonb_build_object('error', 'Event is at full
        capacity');

    END IF;

END

IF;

```

```
INSERT INTO event_tickets (event_id, user_id,  
status)
```

```
VALUES (p_event_id, p_user_id, 'confirmed')
```

```
RETURNING id INTO v_ticket_id;
```

```
RETURN jsonb_build_object(  
  

```

```
    'ticket_id',          v_ticket_id,  
  

```

```
    'status',             'confirmed',  
  

```

```
    'already_registered', FALSE  
  

```

```
);
```

```
END;
```

```
  
$function$;
```

```
-- ----- release_cron_lock -----
```

```
CREATE OR REPLACE FUNCTION
```

```
public.release_cron_lock(p_job_name text)
```

```
RETURNS void
```

```
LANGUAGE plpgsql
```

```
SECURITY DEFINER
```

```
SET search_path TO 'public'
```

```
AS $function$
```

```
    BEGIN
```

```

        DELETE FROM cron_locks WHERE job_name
        = p_job_name;
    END;
    $function$;

-- ----- rls_auto_enable -----
CREATE OR REPLACE FUNCTION public.rls_auto_enable()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE v_table TEXT;
BEGIN
    FOR v_table IN (
        SELECT tablename FROM pg_tables
        WHERE schemaname = 'public'
        AND tablename NOT IN
        ('schema_migrations','spatial_ref_sys')
    ) LOOP
        EXECUTE format('ALTER TABLE public.%I ENABLE
ROW LEVEL SECURITY', v_table);
    END LOOP;
END;
$function$;

-- ----- set_updated_at -----
CREATE OR REPLACE FUNCTION public.set_updated_at()
RETURNS trigger
LANGUAGE plpgsql
SET search_path TO 'public'
AS $function$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$function$;

-- ----- try_acquire_cron_lock -----
CREATE OR REPLACE FUNCTION

```

```

public.try_acquire_cron_lock(p_job_name text)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
BEGIN
    INSERT INTO cron_locks (job_name, locked_at)
    VALUES (p_job_name, now())
    ON CONFLICT (job_name) DO NOTHING;
    RETURN FOUND;
END;
$function$;

-- ----- try_acquire_cron_lock -----
CREATE OR REPLACE FUNCTION
public.try_acquire_cron_lock(lock_name text,
timeout_seconds integer DEFAULT 5)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
    DECLARE
        acquired BOOLEAN;
        lock_id BIGINT;
    BEGIN
        lock_id :=
hashtext('cron_lock_' || lock_name);

        --  CORRECT:
        pg_try_advisory_lock (NOT pg_trylock_advisory_lock)
        SELECT
        pg_try_advisory_lock(lock_id) INTO acquired;

        RETURN
        acquired;

    END;
$function$;

```

```

-- ----- unlock_pending_hp_fifo_atomic -----
--
CREATE OR REPLACE FUNCTION
public.unlock_pending_hp_fifo_atomic(p_user_id
uuid, p_order_id uuid, p_amount_to_unlock integer)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
DECLARE
    v_active_hp INTEGER;
    v_remaining INTEGER := p_amount_to_unlock;
    v_unlocked_total INTEGER := 0;
    r RECORD;
    v_new_pending_amount INTEGER;
BEGIN
    -- Row-lock profiles
    SELECT hp_balance INTO v_active_hp FROM
public.profiles WHERE id = p_user_id FOR UPDATE;
    IF NOT FOUND THEN
        RETURN jsonb_build_object('error', 'User
profile not found');
    END IF;

    -- Lock and process pending transactions (FIFO)
    FOR r IN (
        SELECT id, amount FROM public.hp_transactions
        WHERE user_id = p_user_id AND status =
'pending'
        ORDER BY created_at ASC, id ASC
        FOR UPDATE
    ) LOOP
        IF v_remaining <= 0 THEN
            EXIT;
        END IF;

        IF r.amount <= v_remaining THEN
            -- Fully unlock this transaction
            UPDATE public.hp_transactions SET status =

```



```

'active' WHERE id = r.id;
    v_active_hp := v_active_hp + r.amount;
    v_unlocked_total := v_unlocked_total +
r.amount;
    v_remaining := v_remaining - r.amount;
ELSE
    -- Partially unlock this transaction
    v_new_pending_amount := r.amount -
v_remaining;
    UPDATE public.hp_transactions SET amount =
v_new_pending_amount WHERE id = r.id;

    -- Insert split active transaction
    INSERT INTO public.hp_transactions (
        user_id, amount, type, status,
balance_after, source, reference_type,
reference_id, metadata
    )
    VALUES (
        p_user_id, v_remaining, 'earn', 'active',
v_active_hp + v_remaining, 'unlock', 'order',
p_order_id, jsonb_build_object('notes', 'Pending HP
unlocked (FIFO split)')
    );

    v_active_hp := v_active_hp + v_remaining;
    v_unlocked_total := v_unlocked_total +
v_remaining;
    v_remaining := 0;
END IF;
END LOOP;

IF v_unlocked_total > 0 THEN
    UPDATE public.profiles SET hp_balance =
v_active_hp WHERE id = p_user_id;
END IF;

RETURN jsonb_build_object('success', true,
'unlocked_amount', v_unlocked_total, 'new_balance',
v_active_hp);

```

```
END;  
$function$;
```